

Finite State, Forced Forgetting: Investigating the Memory Mechanism in Sub-quadratic Models

Anonymous ACL submission

Abstract

1 Introduction

2 Related Work

Attention-free sub-quadratic sequence models.

Recent “linear recurrent” language-model backbones replace token–token softmax attention with an explicit recurrent state updated online, enabling linear-time inference and constant per-token memory. This family includes structured state space models and closely related linear-attention-as-recurrence formulations, spanning S4-style layers and their simplifications (Gu et al., 2022a; Smith et al., 2023; Gupta et al., 2022; Gu et al., 2022b), selective SSM architectures such as Mamba (Gu and Dao, 2024), and Transformer/RNN hybrids such as RWKV (Peng et al., 2023). A central theme in improving *memory and recall* in these models is *controlling forgetting* through input-dependent gating or decay (Gu and Dao, 2024; Peng et al., 2023, 2025; Yang et al., 2024). Complementary work increases *effective state capacity* and state-tracking ability—often framed as a key bottleneck for in-context retrieval—by moving from purely additive updates to delta-rule or gated-delta updates and related state-expansion mechanisms (Yang et al., 2025b,a; Siems et al., 2025; Grazzi et al., 2025). Finally, several works refine the SSM layer/algorithm itself (e.g., more stable parameterizations/initializations and dual views that connect SSMs to masked attention), enabling more reliable long-range modeling and more efficient long-sequence training/inference (Gu et al., 2022b; Smith et al., 2023; Dao and Gu, 2024; Fu et al., 2023).

Entity binding as an in-context memory primitive. Entity–attribute binding tasks offer a controlled probe of in-context memory: the model

must maintain multiple concurrent associations (e.g., a key-value dictionary) and retrieve the correct one at query time, without relying on pre-training knowledge. Significant Recent work has emphasized *synthetic* binding and entity-tracking setups—from static multi-entity attribute binding prompts to procedurally generated state-update narratives—that precisely control the number of entities, interference, and context length (Feng and Steinhardt, 2024; Kim and Schuster, 2023; Prakash et al., 2024; Dai et al., 2024). Interpretability studies on these benchmarks suggest that successful LMs allocate dedicated representational degrees of freedom that act like *identifiers* for bindings: Some argue for a “binding ID” subspace attached to entity and attribute token positions (Feng and Steinhardt, 2024); follow-up representational and causal analyses refine this picture by isolating low-rank directions encoding order-related signals that govern binding behavior (Dai et al., 2024). Complementary mechanistic work on synthetic *variable* binding further supports the view that binding can be mediated by sparse, specialized circuits that are discoverable via automated causal localization and patching (Davies et al., 2023; Wu et al., 2025). Our entity-binding datasets follow this controlled regime, but we target sub-quadratic recurrent-state models where bindings must be retained in a compressed state; this shifts the emphasis from *whether the model attends to the correct key-value pair* to *which bindings persist in the state and whether the model’s query mechanism is able to retrieve them*.

Interpretability tools for studying memory mechanisms.

Our work sits within mechanistic interpretability, which aims to localize and characterize internal computations responsible for model behavior. Probing methods train auxiliary predictors to decode information from hidden activations; recent ‘lens’ techniques decode intermediate representations into output distributions, providing a

token-level view of how predictions evolve across depth (Belrose et al., 2025). Beyond correlational probes, causal interventions are commonly used to establish necessity and sufficiency of components. Meng et al. develop causal tracing interventions to localize computations supporting factual recall and show that specific mechanisms can be edited (ROME) (Meng et al., 2023). Activation patching has become a standard causal tool for circuit-level analysis, and recent work systematizes methodological choices and metrics for patching-based localization (Zhang and Nanda, 2024). Broader frameworks for circuit-based reasoning about Transformers provide a language for decomposing computations into interpretable parts (Elhage et al., 2021). We adopt these interpretability principles in the sub-quadratic setting by (i) designing probes that mirror the architecture’s native readout direction and (ii) using counterfactual patching on internal state representations to test whether probe-identified components have a causal effect on retrieval.

3 State Space Models

Many state-of-the-art sub-quadratic architectures, including Mamba [Add citation], RWKV [Add citation], GLA [Add citation], and Deltanet [Add citation], can be unified under the framework of *state space models* (SSMs); see section 3.1 for concrete examples. Despite their architectural differences, all of these models can be written in the following generic form.

The model maintains an internal state $S_t \in \mathbb{R}^{d \times d}$ that is updated at each time step based on the previous state and the current input $x_t \in \mathbb{R}^d$:

$$S_t = f(S_{t-1}) + g(x_t), \quad (1)$$

$$o_t = S_t q(x_t), \quad (2)$$

where f is a decay or transformation function applied to the previous state, $g(x_t)$ is some function of the current input (independent of other timesteps), and $o_t \in \mathbb{R}^d$ is the output. The output is produced by *querying* the state via right-multiplication with a query vector $q(x_t) \in \mathbb{R}^d$ derived from the current input.

Unlike transformers, whose attention mechanism incurs $O(L^2)$ computational cost with respect to sequence length L , SSM-based models enable inference in $O(L)$ time. This efficiency comes from compressing all past information into a fixed-size state $S_t \in \mathbb{R}^{d \times d}$, which is updated incrementally rather than recomputed from scratch.

This raises a fundamental question: given the finite capacity of the state, *how does the model store and retrieve in-context information?*

3.1 Examples

To illustrate the generality of this framework, we show how two popular architectures fit the SSM formulation.

Gated Linear Attention (GLA). In GLA, the state update and query take the form:

$$S_t = S_{t-1} \odot (1\alpha_t^\top) + v_t k_t^\top, \quad o_t = S_t q_t, \quad (3)$$

where $\odot$ denotes element-wise multiplication, and $\alpha_t, v_t, k_t, q_t \in \mathbb{R}^d$ are learned projections of the input.

Mamba-2. In Mamba-2, the formulation is:

$$S_t = \gamma_t S_{t-1} + v_t k_t^\top, \quad o_t = S_t q_t, \quad (4)$$

where $\gamma_t \in \mathbb{R}$ is a learned scalar decay factor.

Both architectures share the key property that the output is produced by querying the state from the right.

4 Methodology

4.1 Entity-Binding Datasets

We construct *in-context* entity-binding datasets to study how SSMs store and retrieve factual associations. Each dataset follows a generic structure: samples consist of N_e statements of the form “<entity> <relation> <property>”, where each entity is associated with one of C possible property values.

These associations are provided entirely within the input prompt (not learned during pretraining) and vary randomly across samples. One designated *target entity* appears in all samples, but its associated property changes. The order of entities is randomized, and the target entity appears at a random position. This ensures that any successful retrieval must rely on in-context memory rather than memorized facts. See fig. 1 for examples.

4.2 Probing State Representations

Recall from eq. (2) that SSMs produce outputs by querying the state via right-multiplication: $o_t = S_t q(x_t)$. This suggests a natural way to probe whether entity-property bindings are stored in the state: we can train a probe that mimics this query mechanism.

Favorite Color ($N_e = 30, C = 10$) <i>Relation:</i> "X's favorite color is Y." Lady Gaga's favorite color is red. Beyoncé's favorite color is green. ...
Lives in City ($N_e = 30, C = 10$) <i>Relation:</i> "X lives in Y." The shark lives in France. The dog lives in Italy. ...

Figure 1: Examples from our entity-binding datasets. Each sample contains N_e entity-property pairs with one target entity whose property varies across samples.

Given the following context, let's answer the question below. Context: Lady Gaga's favorite color is red. Beyoncé's favorite color is green. ... Question: What is Lady Gaga's favorite color? Answer: Lady Gaga's favorite color is

Figure 2: Prompt template for evaluating model retrieval. The model must generate the correct property (e.g., "red") as the next token.

For each internal state $S_t \in \mathbb{R}^{d \times d}$ (indexed by layer ℓ and head h), we extract the state at the *final token position* of the input sequence. We then train a probe with two learned components: (i) a **query component** $q_{\text{probe}} \in \mathbb{R}^d$ that queries the state from the right, producing $S_t q_{\text{probe}} \in \mathbb{R}^d$; and (ii) a **classifier component** $W_{\text{cls}} \in \mathbb{R}^{C \times d}$ that maps the queried vector to C class logits.

The full probe is:

$$\text{probe}(S_t) = W_{\text{cls}} S_t q_{\text{probe}} \in \mathbb{R}^C. \quad (5)$$

We call this the *natural probe* because it mirrors the model's native state-querying mechanism: the state is multiplied by a vector *from the right*, just as in eq. (2).

To validate that this structure is important, we also train a *non-natural probe* that queries from the opposite direction:

$$\text{probe}_{\text{non-nat}}(S_t) = (q_{\text{probe}}^\top S_t) W_{\text{cls}} \in \mathbb{R}^C, \quad (6)$$

where $q_{\text{probe}} \in \mathbb{R}^d$ is applied from the left, and $W_{\text{cls}} \in \mathbb{R}^{d \times C}$ projects to class logits. If memory is encoded consistently with the model's query mechanism, the *natural probe* should significantly outperform the *non-natural probe*; we verify this in section 5.1.

4.3 Evaluating Model Retrieval

To compare probe accuracy with the model's own retrieval ability, we construct a question-answering prompt that queries the model for the target entity's property. fig. 2 shows an example for the Favorite Color dataset.

We measure top-1 accuracy: the model must generate the correct property label as its next token, where we only consider tokens corresponding to valid property values from the dataset (e.g., the $C = 10$ colors).

4.4 Causal Intervention

To establish that the heads identified by probing have a *causal* effect on model behavior, we perform counterfactual patching experiments using the prompt structure from section 4.3.

The procedure is as follows: (i) generate two prompts A and B that differ only in the target entity's property (e.g., "Lady Gaga's favorite color is red" vs. "Lady Gaga's favorite color is blue"); (ii) run both prompts through the model, collecting states after processing the context; (iii) for prompt A , replace the states of a *subset* of heads with the corresponding states from prompt B ; and (iv) continue generation and measure whether the output switches from property A to property B .

We compare several head selection strategies: (i) **Baseline**: no heads patched, measuring the model's normal accuracy; (ii) **Natural probe heads**: the top-ranked heads by *natural probe* accuracy; (iii) **Non-natural probe heads**: the top-ranked heads by *non-natural probe* accuracy; and (iv) **All heads**: replacing all head states, representing the maximum possible effect.

5 Results

We conduct all experiments using RWKV-7 1.5B, an SSM-based language model with 24 layers and 32 heads per layer (768 total heads), achieving state-of-the-art performance at its scale [Add citation].

5.1 Validating the Probing Approach

Before interpreting probe results, we first verify that the probe architecture is meaningful: it should only succeed when the target information is actually encoded in the state, rather than being powerful enough to extract arbitrary patterns.

Table 1: Top 10 heads ranked by *natural probe* validation accuracy on the Favorite Color dataset.

Rank	Head	Val Acc.	Train Acc.
1	L15H11	96.3%	97.5%
2	L15H2	93.0%	95.4%
3	L15H13	91.4%	93.7%
4	L14H23	88.5%	91.4%
5	L15H27	79.0%	81.1%
6	L9H18	72.8%	76.6%
7	L15H1	65.3%	67.9%
8	L14H18	61.8%	65.9%
9	L15H8	61.1%	62.8%
10	L12H18	58.6%	63.3%

5.1.1 The Probe is Not Overly Powerful

If the probe were too expressive, it could achieve high accuracy on any head regardless of whether that head stores relevant information. However, table 1 shows that the *natural probe* achieves high accuracy on only a small fraction of heads. The top head (L15H11) reaches 96.3%, but accuracy drops sharply: by rank 5 it falls below 80%, and by rank 10 below 60%. Most of the 768 heads yield near-chance performance.

This concentration of high-accuracy probes in a small subset of heads, rather than diffuse success across all 768 heads, indicates that the probe is identifying genuine “memory heads” rather than exploiting spurious correlations.

5.1.2 Querying from the Right Outperforms Querying from the Left

The *natural probe* (querying from the right) significantly outperforms the *non-natural probe* (querying from the left): the best *natural probe* reaches 96.3%, while the best *non-natural probe* achieves only 70.8%.

This gap confirms that memory is encoded in a manner consistent with the model’s native query mechanism (eq. (2)). Probing from the right extracts information effectively; probing from the left does not. fig. 3 visualizes this comparison across the top 10 heads for each probe type.

5.2 Causal Effect of Memory Heads

To verify that probe-identified heads play a causal role in storing memory, we perform counterfactual patching experiments on 50 sample pairs from the Favorite Color dataset. table 2 shows the results.

Without patching (Baseline), the model outputs the correct property 90% of the time. When we patch the top 0.5% of heads selected by *natural*

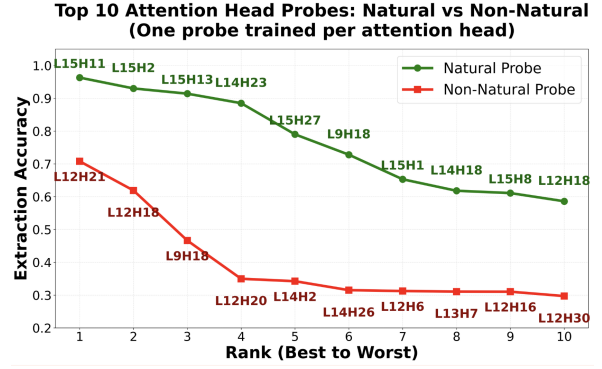


Figure 3: Comparison of *natural probe* (querying from the right) versus *non-natural probe* (querying from the left) accuracy on the Favorite Color dataset. Each point represents the best probe for a given head, ranked by accuracy. The natural probe consistently outperforms the non-natural probe, with the gap widening at higher ranks.

Table 2: Counterfactual patching results (50 pairs). “Correct”: original property; “Switched”: counterfactual property; “Neither”: other output.

Method	Correct	Switched	Neither
Baseline	90%	0%	10%
<i>Natural</i> (top 0.5%)	14%	36%	50%
<i>Non-natural</i> (top 0.5%)	44%	4%	52%
All heads	0%	76%	24%

probe accuracy, the model switches to the counterfactual property 36% of the time. In contrast, patching the same number of heads selected by *non-natural probe* accuracy yields only a 4% switch rate. Patching all heads provides an upper bound, achieving 76% switch rate. [TODO: add experiment with random head selection as additional baseline]

5.3 The Bottleneck is Querying, Not Storage

A key observation emerges from comparing probe performance to model performance. While our probes extract the correct binding with up to 96.3% accuracy, the model’s own ability to answer questions about these bindings is substantially lower.

When we prompt the model using the template in fig. 2, its accuracy is far below probe accuracy. This gap implies that the information *is stored* in the state (the probe finds it) but the model struggles to *query* it effectively during generation.

This finding challenges the common assumption that SSMS’ limitations stem from insufficient memory capacity. Instead, our results suggest that the bottleneck lies in the model’s ability to formulate

Top 10 Memory Heads By Dataset

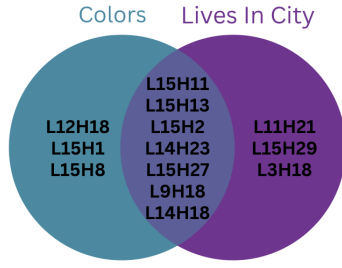


Figure 4: Overlap of top 10 heads ranked by *natural probe* accuracy across two different datasets. Seven heads (shown in the intersection) achieve high accuracy on both Favorite Color and Lives in City tasks, demonstrating domain-general memory storage capabilities.

effective queries.

5.4 Universal Memory Heads

We analyze whether the same heads are important across different semantic domains by comparing top-performing heads on the Favorite Color dataset versus Lives in City.

We find substantial overlap: **7 of the top 10 heads are shared across both datasets**. fig. 4 visualizes this overlap, showing that the majority of top-performing heads are consistent across semantically distinct tasks. This suggests the existence of *universal memory heads* that specialize in entity-property binding regardless of semantic content. We defer investigation of the mechanism underlying these heads to future work.

5.5 Semantic Collision Accelerates Forgetting

We investigate how context affects memory retention by measuring probe accuracy as a function of token distance from the information. We observe that **increased semantic similarity between the prompt content and the target subject accelerates the model’s forgetting rate** as can be seen in figs. 5 and 6. When subsequent sentences discuss similar entities, probe accuracy degrades faster than when the intervening content is semantically distinct. More specifically, we create two types of datasets: a homogeneous dataset containing only sentences of one type (e.g., "Lady Gaga’s favorite animal is Fox"), and a mixed dataset that combines sentences from two different domains (e.g., favorite animals and favorite colors) in a ratio where the target domain represents only a small fraction (roughly 1/30) of the total sentences.

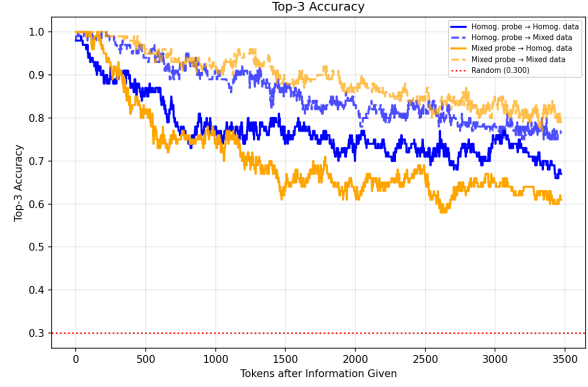


Figure 5: Top 3 accuracy of the homogeneous and mixed data probes on homogeneous and mixed data as a function of token distance from the binding pair we query.

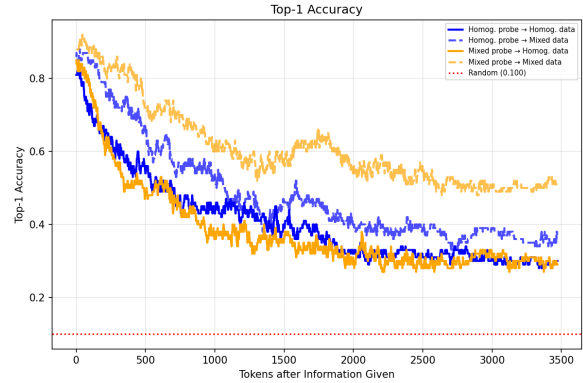


Figure 6: Top 1 accuracy of the homogeneous and mixed data probes on homogeneous and mixed data as a function of token distance from the binding pair we query.

6 Future Work

Our findings open several directions for future research: (i) **Characterizing Universal Memory Heads**: what makes these heads special, and could understanding their mechanism inform architectural improvements? (ii) **Improving State Querying**: given that the bottleneck appears to be querying rather than storage, can we design better query mechanisms? (iii) **Scaling Analysis**: do these patterns hold in larger models?

7 Conclusion

We investigated the in-context memory mechanism of state space models by probing their internal states. Our key findings are: (i) **Natural probing works**: probes that query the state from the right significantly outperform probes that query from the left; (ii) **Memory is localized**: only a small subset of heads store entity-property bindings with high fidelity; (iii) **Probing identifies causal**

heads: counterfactual patching confirms that probe-identified heads have genuine causal influence; (iv) **The bottleneck is querying:** probe accuracy far exceeds model accuracy, suggesting information is stored but poorly retrieved; (v) **Universal memory heads exist:** the same heads are important across different semantic domains; and (vi) **Semantic collision hurts:** similar semantic content accelerates forgetting.

These results suggest that improving sub-quadratic models may require focusing on the query mechanism rather than storage capacity.

A Probing Training Details

Training configuration for the probing experiments: train/validation split of 70%/30%, maximum of 20,000 epochs, early stopping patience of 20 epochs, batch size of 1,000, and learning rate of 10^{-3} with Adam optimizer.

References

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. 2025. [Eliciting latent predictions from transformers with the tuned lens](#).

Qin Dai, Benjamin Heinzerling, and Kentaro Inui. 2024. [Representational analysis of binding in language models](#).

Tri Dao and Albert Gu. 2024. [Transformers are ssms: Generalized models and efficient algorithms through structured state space duality](#).

Xander Davies, Max Nadeau, Nikhil Prakash, Tamar Rott Shaham, and David Bau. 2023. [Discovering variable binding circuitry with desiderata](#).

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. 2021. A mathematical framework for transformer circuits.

Jiahai Feng and Jacob Steinhardt. 2024. [How do language models bind entities in context?](#)

Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. 2023. [Hungry hungry hippos: Towards language modeling with state space models](#).

Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. 2025. [Unlocking state-tracking in linear rnns through negative eigenvalues](#).

Albert Gu and Tri Dao. 2024. [Mamba: Linear-time sequence modeling with selective state spaces](#).

Albert Gu, Karan Goel, and Christopher Ré. 2022a. [Efficiently modeling long sequences with structured state spaces](#).

Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré. 2022b. [On the parameterization and initialization of diagonal state space models](#).

Ankit Gupta, Albert Gu, and Jonathan Berant. 2022. [Diagonal state spaces are as effective as structured state spaces](#).

Najoung Kim and Sebastian Schuster. 2023. [Entity tracking in language models](#).

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2023. [Locating and editing factual associations in gpt](#).

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. 2023. [Rwkv: Reinventing rnns for the transformer era](#).

Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaxing Liu, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, Nathan Wilce, Johan S. Wind, Tianyi Wu, Daniel Wuttke, and Christian Zhou-Zheng. 2025. [Rwkv-7 "goose" with expressive dynamic state evolution](#).

Nikhil Prakash, Tamar Rott Shaham, Tal Haklay, Yonatan Belinkov, and David Bau. 2024. [Fine-tuning enhances existing mechanisms: A case study on entity tracking](#).

Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. 2025. [Deltaproduct: Improving state-tracking in linear rnns via householder products](#).

Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. 2023. [Simplified state space layers for sequence modeling](#).

Yiwei Wu, Atticus Geiger, and Raphaël Millièvre. 2025. [How do transformers learn variable binding in symbolic programs?](#)

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025a. [Gated delta networks: Improving mamba2 with delta rule](#).

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. 2024. [Gated linear attention transformers with hardware-efficient training](#).

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen,
and Yoon Kim. 2025b. [Parallelizing linear transformers with the delta rule over sequence length.](#)

Fred Zhang and Neel Nanda. 2024. [Towards best practices of activation patching in language models: Metrics and methods.](#)